Playing Mastermind with Answer Set Programming: scaling of a consistent-guess strategy with and without repeated colors

Kyana Marckx

November 2026

Contents

1 Introduction **1**

2 Background **2**
2.1 The game 2
2.2 Strategies 2
2.3 ASP and clingo 2

3 Approach **2**
3.1 Models 2
3.2 Strategy 3

4 Results **3**
4.1 Validation (RQ1) 3
4.2 Without repetition (RQ2) 3
4.3 With repetition and encoding comparison (RQ3, RQ4) 4

5 Discussion and limitations **4**
5.1 Limitations 5

6 Conclusion **5**

Abstract

Mastermind is a code-breaking game in which a codebreaker guesses a hidden color code and receives red (or black, depending on the specific game variant) and white peg feedback. In this report, the codebreaker is modeled in Answer Set Programming (ASP) with clingo: the answer sets are exactly the codes consistent with all feedback so far, and the strategy plays the first answer set found. Codes of four positions are studied with 4 to 10 colors, with and without repeated colors, under a limit of 12 guesses. For the variant with repetition, two encodings of the feedback are compared: one based on `#min`, and one based on unary occurrence indexing. Both were validated against a Python reference. Without repetition, the average number of guesses grows from 3.58 to 5.62 while one round of grounding and solving stays at about one millisecond. With repetition, guesses grow faster (6.57-6.69 at 10 colors), and the occurrence encoding yields a 34-39% smaller ground program and 12-22% lower run time than the `#min` encoding. Every game was solved within 12 guesses.

1 Introduction

Mastermind is originally a two-player code-breaking game: a codemaker hides a code of colored pegs, and a codebreaker tries to identify it through guesses and feedback. The game has been analyzed extensively [Knu77, Irv79, KL93, Koo05] and is computationally hard, since deciding whether a code exists that is consistent with a list of guesses and feedback is NP-complete [SZ06]. This makes it a natural test case for ASP, a declarative paradigm for combinatorial search problems [BET11]. The rules are easy to state: a code is a candidate exactly when it would have produced the same feedback for earlier guesses. The difficulty lies in the white pegs, which require counting

color overlaps without counting a peg twice. Several ASP encodings of this count are possible and differ in the size of the resulting ground program.

The research question is: "How do playing strength and computational cost of a consistent-guess Mastermind strategy implemented in ASP scale with the number of colors k for codes of four positions, and how do the rules (repetition of colors allowed or not) and the encoding of the feedback influence this?". Four sub-questions are addressed: (RQ1) are the encodings correct; (RQ2) how do guesses and run time grow with k without repetition; (RQ3) what changes when repetition is allowed; (RQ4) how do two feedback encodings compare in ground size and run time.

The main variant forbids repeated colors, following the edition of the official instructions consulted [Has]. The literature mostly studies the variant with repetition [Knu77], so both are covered. The codemaker is passive, ie. it holds a fixed secret code. Optimal strategies and adversarial codemakers are outside the scope.

2 Background

2.1 The game

A secret code $c = (c_1, ..., c_p)$ assigns one of k colors to each of p positions. Each guess $g = (g_1, ..., g_p)$ is answered with two numbers. The red/black pegs are $r(g, c) = |i : g_i = c_i|$. The total number of color matches is $m(g, c) = \sum_x min(|g_x|, |c_x|)$, where $|g_x|$ is the number of occurrences of color x in g, and the white pegs are $w(g, c) = m(g, c) - r(g, c)$. Without repetition, $m(g, c)$ is simply the number of colors shared by g and c. A code is consistent with earlier guesses if it would have received the same feedback for each of them [MGMCR12]. The classical game has $p = 4$ and $k = 6$; with repetition there are k^p codes (=1296), without repetition $k!/(k - p)!$ (=360). The game was invented by Meirowitz in 1970 [BGL09].

2.2 Strategies

For $p = 4$ and $k = 6$ with repetition, Knuth [Knu77] obtained 4.478 expected guesses with a worst case of five, Koyama and Lai [KL93] found the optimal 4.340 by exhaustive search, and Kooi [Koo05] reached 4.373 with a simple heuristic. Playing any consistent code is the simplest strategy; playing the first consistent code found by a search is statistically indistinguishable from random consistent play [MGMCR12], for which Weisstein [Wei] reports 4.638 guesses. Without repetition and with 10 digits (the game Bulls and Cows), Tanaka [Tan96] computed an optimal 26274/5040 $\approx$ 5.213 guesses.

2.3 ASP and clingo

An ASP program consist of rules `head :- body.`. Its answer sets are the stable models in the sense of Gelfond and Lifschitz [GL88]. Facts are rules without body (`head.`), constraints (`:- body.`) eliminate answer sets, choice rules such as `1 { code(P,C) : color(C) } 1 :- position(P).` let the solver choose, and aggregates such as `#count`, `#sum`, and `#min` compute values over sets of atoms [GKKS12]. Programs typically follow a generate-define-test pattern. Solving has two phases: a grounder replaces variables by constants, after which a solver searches for answer sets [GKK+11]; either phase can dominate run time. Clingo supports multi-shot solving, in which programs are extended step by step through an API, only the new parts are grounded, and the solver can reuse what it learned earlier [GKKS19]. This fits a game in which each turn adds one guess and one feedback.

3 Approach

3.1 Models

Both variants share a generating choice rule. Without repetition, a second rule forces each color to be used at most once (k^p and $k!/(k-p)!$ answer sets, respectively; 1296 and 360 were confirmed for $k = 6$).

```
position(1..p). color(1..k).
1 { code(P,C) : color(C) } 1 :- position(P).
{ code(P,C) : position(P) } 1 :- color(C).
```

After each guess, a program part for turn t is added, with t inserted as a constant. It contains the facts `guess(t,P,C)` and `fb(t,R,W)` and the following rules.

```
red(t,N) :- N = #count{ P : guess(t,P,C), code(P,C) }.
:- fb(t,R,_), red(t,N), N != R.
:- fb(t,R,W), common(t,N), N != R+W.
```

The predicate `common(t,N)` holds if N is the total number of color matches. Without repetition it is defined by `common(t,N) :- N = #count{ C : guess(t,_,C), code(_,C) }.`. For the variant with repetition, two encodings are compared.

Encoding A translates the definition literally using `#min`:

```
nc(C,N) :- color(C), N = #count{ P : code(P,C) }.  % base part
ng(t,C,N) :- color(C), N = #count{ P : guess(t,P,C) }.
m(t,C,M) :- ng(t,C,A), nc(C,R), M = #min{ A ; R }.
common(t,S) :- S = #sum{ M,C : m(t,C,M) }.
```

Encoding B represents counts in unary: `cocc(C,I)` states that the code contains color C at least I times, and `gocc(t,C,I)` that the guess does.

```
cocc(C,I) :- color(C), I = 1..p, I <= #count{ P : code(P,C) }.  % base part
gocc(t,C,I) :- color(C), I = 1..p, I <= #count{ P : guess(t,P,C) }.
common(t,N) :- N = #count{ C,I : gocc(t,C,I), cocc(C,I) }.
```

B is correct because $min(|g|_x, |c|_x)$ equals the number of indices $I >= 1$ with $I <= |c|_x$. The expectation, formulated before experiments, was that B yields a smaller ground program, because it needs no extra aggregate per color.

3.2 Strategy

A python driver plays complete games using clingo's multi-shot API. A fixed first guess is played ((1,2,3,4) without repetition, (1,1,2,2) with repetition, the latter being one of the five non-equivalent openings listed by Koyama and Lai [KL93]). After each unsuccessful guess, the feedback is computed by a Python reference function, the turn program is added and grounded, and the solver is asked for a single answer set, which becomes the next guess. A game is a success if the code is found within 12 guesses.

4 Results

4.1 Validation (RQ1)

Both encodings returned the same number of answer sets as the reference in all comparisons, and every game ended with the secret code. This is evidence of correctness for the tested instances, not a proof.

4.2 Without repetition (RQ2)

All games were solved within 12 guesses (see Table 1).

k	Games	Avg guesses	Max	Time/game (ms)
4	24	3.58	6	2.70
5	120	3.95	6	3.13
6	360	4.16	6	3.25
7	840	4.53	8	3.69
8	1680	4.91	8	6.64
9	300 of 3024	5.20	7	4.52
10	300 of 5040	5.62	8	5.01

Table 1: Results without repetition ($p = 4$).

The average number of guesses grows by about 0.34 per additional color, while the number of codes grows by a factor of 210 (24 to 5040). The time per game increases from 2.7ms at $k = 4$ to 5.0ms at $k = 10$, apart from the marked value for $k = 8$ (likely due to external background process on the laptop). Grounding takes 53-54% of the time.

4.3 With repetition and encoding comparison (RQ3, RQ4)

Again, all games were solved within 12 guesses (see Table 2).

k	Enc.	Games	Avg guesses	Max	Ground (ms)	Solve (ms)	Rules	Atoms
4	A	256	3.73	6	2.27	1.58	300.91	256.44
4	B	256	3.71	6	1.93	1.46	198.61	185.14
5	A	625	4.23	7	2.71	1.92	368.81	310.96
5	B	625	4.20	6	2.31	1.69	231.64	218.48
6	A	1296	4.72	7	3.23	2.26	439.51	366.44
6	B	1296	4.71	7	2.66	1.95	264.56	251.81
7	A	284	5.14	8	3.67	2.57	509.25	419.21
7	B	284	5.20	8	3.06	2.19	296.27	284.12
8	A	289	5.65	8	4.18	3.02	585.84	476.82
8	B	289	5.67	9	3.46	2.50	324.55	313.68
9	A	291	6.19	9	4.77	3.42	670.04	539.35
9	B	291	6.13	9	3.82	2.78	354.31	344.27
10	A	296	6.69	10	5.37	3.96	755.72	601.38
10	B	296	6.57	10	4.21	3.04	382.52	373.56

Table 2: Results with repetition ($p = 4$)

Encoding B yields a consistently smaller ground program than A: 34% fewer rules at $k = 4$, rising to 49% at $k = 10$, and 28-38% fewer atoms. Total run time is 12-22% lower, with grounding 15-22% and solving 8-23% lower; the advantage grows with k. This agrees with the expectation. Encoding A needs about 76 additional rules per extra color, encoding B about 31, although this is confounded by the growing number of rounds per game. Grounding accounts for 57-59% of the time in both encodings. The average number of guesses of A and B differ by at most 0.12. For $k \leq 6$, where all codes are played, these are exact differences; for sampled k the paired mean difference (B minus A) is +0.060, +0.024, -0.062, and -0.122 for $k = 7...10$, each within 1.6 standard errors of zero (standard errors 0.057-0.078). Both encodings define the same consistent codes, so such differences stem from different first answer sets and show no evidence of a systematic effect.

Allowing repetition increases the number of guesses (see Figure 1 and Figure 2).

k	Avg guesses: no rep	A	B
4	3.58	3.73	3.71
5	3.95	4.23	4.20
6	4.16	4.72	4.71
7	4.53	5.14	5.20
8	4.91	5.65	5.67
9	5.20	6.19	6.13
10	5.62	6.69	6.57

Figure 1: Variant without repetition versus encodings A and B with repetition: average guesses

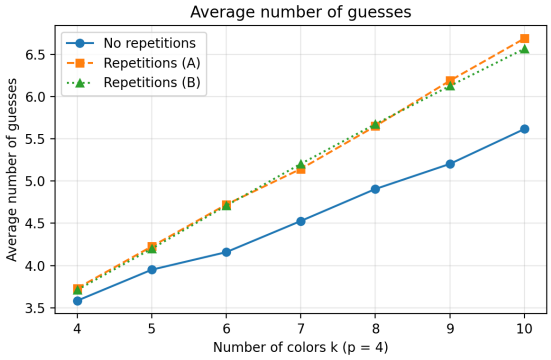


Figure 2: Average number of guesses with ascending number of colors

The growth per extra color is about 0.34 guesses without repetition and about 0.48-0.49 with repetition, and the gap between the variants widens from 0.13-0.15 guesses at $k = 4$ to 0.95-1.07 at $k = 10$. Part of this is the larger code space (for example 10000 versus 5040 codes at $k = 10$), and the comparison is not like-for-like because the first guess also differs.

5 Discussion and limitations

For four positions and up to ten colors, ASP is not a computational bottleneck for a consistent-guess strategy: a round costs about 1.0-1.1ms without repetition and 1.2-1.6ms with repetition, nearly independent of te size of the code space. A plausible explanation is that the ground programs are small and the solver quickly finds a first answer set, so fixed overheads dominate; this was not

profiled and remains a hypothesis. The NP-completeness of the underlying problem does not show in this range, and the extrapolation to larger p or k is not warranted. The unary encoding B is smaller and faster than the literal `#min` translation, and its cost per round hardly grows with k, whereas that of A does; at these sizes the difference is a matter of milliseconds. The strategy is simple but not strong; stronger strategies score candidates by how they partition the consistent set [Koo05, MGMCR12], which requires reasoning over all pairs of codes and is considerably more expensive.

5.1 Limitations

The results are subject to several limitations.

1. All timings come from a single run per variant on one machine at millisecond resolution; repeated measurements with medians were not performed. Run-to-run variation is not negligible: the per-game times of the reported run without repetition differed by between -19% and +6% from those of an earlier execution at the same experiment ($k = 8$ excluded), whereas for repetition with $k = 4...8$ and earlier execution differed by less than 3%. In the reported run without repetition, about a fifth of the games for $k = 8$ were three to five times slower, presumably owing to a transient slowdown of the machine, which inflates that mean by roughly 65% relative to the earlier execution (6.64 versus 4.02ms); this value is marked and excluded from the ranges given. The grounding of the base program is excluded from the timings.

2. For $k \geq 9$ without and $k \geq 7$ with repetition, sample means of 284-300 codes are reported with standard errors of 0.06-0.08 guesses; sampled maxima are lower bounds.

3. The first model depends on clingo's search order, which is not uniformly random; the averages describe this solver, version, and configurations.

4. First guesses differ between variants and from the literature, and no formal significance tests were performed.

5. With p fixed at four and at most ten guesses needed, the limit of 12 guesses never bound, although it is approached at $k = 10$ with repetition; where it starts to bind remains unknown.

6. Only a passive codemaker and one solver were considered.

7. Validation covers small instances and random histories.

6 Conclusion

A consistent-guess Mastermind codebreaker was modeled in ASP, validated against a reference implementation, and measured for four positions and up to ten colors. Without repetition, the average number of guesses grows from 3.58 to 5.62, about 0.34 per color, while a round of grounding and solving stays roughly one millisecond. Allowing repetition raises the growth to about 0.5 guesses per color (6.57-6.69 guesses at ten colors) and makes a round 19-60% more expensive, depending on the encoding. An occurrence-indexing encoding of the feedback produces a 34-49% smaller ground program and 12-22% lower run time than an encoding based on `#min`, with an advantage that grows with the number of colors. Every game was solved within 12 guesses, with averages roughly 8-11% above the best published values. Future work could address an adversarial codemaker, optimization-based guess selection with `#minimize`, larger p and k (including the range where the 12-guess limit binds), other solver configurations, and repeated measurements with medians and confidence intervals.

References

[BET11] Gerhard Brewka, Thomas Eiter, and Mirosław Truszczyński. Answer set programming at a glance. *Communications of the ACM*, 54(12):92–103, 2011.

[BGL09] Lotte Berghman, Dries Goossens, and Roel Leus. Efficient solutions for Mastermind using genetic algorithms. *Computers & Operations Research*, 36(6):1880–1885, 2009.

[GKK+11] Martin Gebser, Roland Kaminski, Benjamin Kaufmann, Max Ostrowski, Torsten Schaub, and Marius Schneider. Potassco: The Potsdam answer set solving collection. *AI Communications*, 24(2):107–124, 2011.

[GKKS12] Martin Gebser, Roland Kaminski, Benjamin Kaufmann, and Torsten Schaub. *Answer Set Solving in Practice*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool, 2012.

[GKKS19] Martin Gebser, Roland Kaminski, Benjamin Kaufmann, and Torsten Schaub. Multi-shot ASP solving with clingo. *Theory and Practice of Logic Programming*, 19(1):27–82, 2019.

[GL88] Michael Gelfond and Vladimir Lifschitz. The stable model semantics for logic programming. In *Proceedings of ICLP-88*, pages 1070–1080, 1988.

[Has] Hasbro. Mastermind game instructions (en-in edition). https://instructions.hasbro.com/api/download/44220_en-in_mastermind.pdf.

[Irv79] Robert W. Irving. Towards an optimum Mastermind strategy. *Journal of Recreational Mathematics*, 11(2):81–87, 1979.

[KL93] Kenji Koyama and Tony W. Lai. An optimal Mastermind strategy. *Journal of Recreational Mathematics*, 25(4):251–256, 1993.

[Knu77] Donald E. Knuth. The computer as Master Mind. *Journal of Recreational Mathematics*, 9(1):1–6, 1977.

[Koo05] Barteld Kooi. Yet another Mastermind strategy. *ICGA Journal*, 28(1):13–20, 2005.

[MGMCR12] Juan-J. Merelo-Guervós, Antonio M. Mora, Carlos Cotta, and Thomas P. Runarsson. An experimental study of exhaustive solutions for the Mastermind puzzle, 2012.

[SZ06] Jeff Stuckman and Guo-Qiang Zhang. Mastermind is NP-complete. *INFOCOMP Journal of Computer Science*, 5:25–28, 2006.

[Tan96] Tetsuro Tanaka. An optimal MOO strategy, 1996.

[Wei] Eric W. Weisstein. Mastermind. MathWorld – A Wolfram Web Resource, https://mathworld.wolfram.com/Mastermind.html.